

Contents

Topics to learn.....	4
1. Gradle project structure	4
2. Plain Java → Spring → Spring Boot	4
3. Spring Environment.....	4
4. Spring Bean Lifecycle (VERY IMPORTANT)	4
5. Configuration styles (CORE SKILL)	5
6. Dependency Injection patterns (DESIGN LEVEL)	5
7. Spring Boot auto-configuration (THE MAGIC)	5
8. Profiles + ConfigurationProperties (REAL WORLD)	6
9. Logging & Observability (ESSENTIAL)	6
10. Exception handling & Validation	6
11. Web layer (after foundations).....	7
12. Data access (JPA/JDBC)	7
13. Testing (VERY IMPORTANT).....	7
Plain Java	8
College class (simple POJO).....	8
Main class (YOU manage everything)	8
No Spring means:	9
Coupling	9
Factory-style Java	10
Why plain Java does NOT scale well	10
Interview	10
Spring framework.....	11
Example.....	11
package	11
Import Statements	11
Class declaration	12
main method	12
Creating Spring Container	12
Getting a bean from Spring.....	13
Calling business logic.....	13
Execution flow	13
Spring Boot.....	14
Your CURRENT code (Plain Spring).....	14

Above application using Spring Boot	14
Main Class	14
College class (almost same)	15
What changed?	15
What @SpringBootApplication really doing?.....	15
BENEFITS of Spring Boot	16
How to EXECUTE Spring Boot app.....	17
Project structure (Spring Boot way).....	17
Interview	17
When should YOU use Spring Boot?	17
Environment in Spring.....	18
What Environment actually is?	18
Why Spring needs Environment	18
What can you do with Environment?	19
1. Read configuration properties	19
2. Check which profile is active	19
3. Provide default values safely.....	19
Why not just use @Value?	20
✓ Database switching	20
✓ Feature flags.....	20
✓ Logging config	20
How Spring populates Environment.....	21
Spring Boot vs Spring Framework	21
When SHOULD you use Environment?.....	21
When you should NOT	21
High level comparision	22
Hand on – Exercise to do	23
Gradle build file.....	23
application.properties (BASE configuration).....	24
application-dev.yml (DEV profile)	24
application-prod.yml (PROD profile).....	24
Main application class.....	24
Environment reader bean (MOST IMPORTANT PART).....	25
Run scenarios (THIS IS WHERE YOU LEARN)	25
Property resolution order (MEMORIZE THIS)	27
REST API - Application	28

Create a Spring Boot project	28
Required dependency	28
Project structure (IMPORTANT)	28
Main Application class	29
Business logic (same as before)	29
REST Controller (NEW)	30
What annotations mean (VERY IMPORTANT).....	30
How execution works (FLOW).....	30
How to RUN the application	31
How to TEST the REST API.....	31
Why REST API is better than <code>main()</code> call.....	31
Interview one-liner.....	31
What next.....	32
DB Access	33
Layers in accessing a Database	33
Handle DB connection.....	33
Package Structure	33

Topics to learn

1. Gradle project structure

- ✓ `src/main/java`, `resources`, `build.gradle`
 - ✓ Tasks, dependencies, lifecycle
- This is **build-time understanding**

2. Plain Java → Spring → Spring Boot

- ✓ Object creation (`new`)
- ✓ IoC / DI
- ✓ `@Component`, `@Configuration`
- ✓ Auto-configuration

This is **runtime architecture understanding**

3. Spring Environment

- ✓ `application.properties` / `yaml`
- ✓ Profiles
- ✓ Property resolution order

This is **externalized configuration understanding**

4. Spring Bean Lifecycle (VERY IMPORTANT)

When exactly is my object created, initialized, destroyed?

As part of this, you will learn

- Bean creation phases
- `@PostConstruct`
- `@PreDestroy`
- `InitializingBean`
- `BeanPostProcessor`
- Lazy vs eager beans

Because:

- Environment is read **before beans**
- Auto-configuration depends on lifecycle

5. Configuration styles (CORE SKILL)

Understand **3 ways of configuring Spring**:

1. Java Config (@Configuration)
2. Component scanning
3. Auto-configuration (Boot magic)

Focus on:

```
@ConditionalOnProperty  
@ConditionalOnClass  
@Profile
```

This connects **Environment** → **Bean creation**

6. Dependency Injection patterns (DESIGN LEVEL)

Learn **how beans should depend on each other**:

- ✓ Constructor injection (best practice)
- ✗ Field injection (why it's bad)
- ✓ Qualifiers
- ✓ Primary beans

This is where **clean architecture** starts.

7. Spring Boot auto-configuration (THE MAGIC)

This is the **most misunderstood topic**.

Learn:

- `spring.factories`
- `AutoConfiguration.imports`
- How Boot decides **what to create**
- Why adding a dependency changes behavior

This explains *why adding `spring-boot-starter-data-jpa` suddenly creates a `DataSource`*

8. Profiles + ConfigurationProperties (REAL WORLD)

You already touched `Environment`, now master:

- `@ConfigurationProperties`
- `@Validated`
- Profile-specific configs:
 - `application-dev.yml`
 - `application-prod.yml`
 -

This is **enterprise-grade configuration**.

9. Logging & Observability (ESSENTIAL)

Learn:

- SLF4J
- Logback
- Log levels per profile
- MDC
- Structured logging

Why?

Logs are your **first debugger in production**

10. Exception handling & Validation

Learn:

- `@ControllerAdvice`
- Global exception handling
- Bean validation (`@NotNull`, `@Size`)
- Custom exceptions

This separates **toy apps** from **production apps**.

11. Web layer (after foundations)

Only now move to:

- Spring MVC
- REST controllers
- Filters vs Interceptors
- Request lifecycle

Why later?

Web is just **one entry point** to Spring.

12. Data access (JPA/JDBC)

Learn:

- Transaction boundaries
- `@Transactional`
- Entity lifecycle
- Lazy vs eager fetching

13. Testing (VERY IMPORTANT)

Learn:

- Unit vs integration tests
- `@SpringBootTest`
- Slice tests
- Mocking beans

Plain Java

College class (simple POJO)

```
package com.testing.qe;

public class College {

    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

If you look at above Java code

- No annotations
- No framework
- Just a normal Java class

Main class (YOU manage everything)

```
package com.testing.qe;

public class TestApplication {

    public static void main(String[] args) {

        // YOU create the object
        College college = new College();

        // YOU call the method
        college.principalName();
    }
}
```

No Spring means:

Responsibility	Who does it now?
Object creation	YOU
Dependency management	YOU
Lifecycle control	YOU
Configuration	YOU
Scalability support	×
Loose coupling	×

Coupling

With out Spring : Tight coupling

```
public class College {  
    private Principal principal = new Principal(); // hard-coded  
    public void showPrincipal() {  
        principal.name();  
    }  
}
```

Problems:

- Cannot replace `Principal`
- Hard to test
- Code changes needed everywhere

With Spring : Loose coupling

```
@Component  
public class College {  
    @Autowired  
    private Principal principal;  
}
```

Spring:

- Creates `Principal`
- Injects it
- Allows easy replacement

Factory-style Java

Before Spring existed, people did this

```
public class ObjectFactory {  
  
    public static College getCollege() {  
        return new College();  
    }  
}
```

Usage:

```
College college = ObjectFactory.getCollege();
```

Spring is basically a **VERY powerful, configurable factory**

Why plain Java does NOT scale well

Imagine:

- 50 classes
- Each depends on 5 others

Without Spring:

- 250+ `new` statements
- Tight coupling
- Nightmare to maintain

With Spring:

- Zero `new`
- Configuration-based wiring

Interview

- ✓ Small utility programs
- ✓ Simple learning demos
- ✓ One-class applications

- ✗ Enterprise apps
- ✗ REST APIs
- ✗ Database-heavy systems

Spring framework

Example

```
package com.testing.qe;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
        College college = context.getBean("college", College.class);
        college.principalName();
    }
}
```

package

package com.testing.qe;

- This defines the **package (namespace)** of the class.
- It helps:
 - Organize code
 - Avoid class name conflicts
 - Enable **component scanning** in Spring

Import Statements

import org.springframework.context.ApplicationContext;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

ApplicationContext	AnnotationConfigApplicationContext
• Central Spring IoC container • Responsible for: ◦ Creating objects (beans) ◦ Managing their lifecycle ◦ Injecting dependencies Think of it as Spring's factory + manager.	• A concrete implementation of <code>ApplicationContext</code> • Used when: ◦ Configuration is done using annotations ◦ No <code>applicationContext.xml</code> is used This class: • Reads <code>@Configuration</code> • Scans <code>@Component</code> • Handles <code>@Bean</code>, <code>@Autowired</code>, etc.

Class declaration

```
public class Test {
```

This is a **plain Java class**, not managed by Spring.

⚠️ Important:

- `Test` itself is **not a Spring bean**
- It just **boots the Spring container**

main method

```
public static void main(String[] args) {
```

- Java application entry point
- Starts Spring manually (not Spring Boot)

Creating Spring Container

```
ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
```

What happens here step-by-step:

1. Spring container is created
2. `CollegeConfig.class` is loaded

```
@Configuration

@ComponentScan("com.testing.qe")

public class CollegeConfig {

}
```

3. Spring:

- Reads `@Configuration`
- Scans packages for:
 - `@Component`
 - `@Service`
 - `@Repository`
 - `@Controller`
- Creates beans

At this moment, all beans are created and stored in the container.

Getting a bean from Spring

```
College college = context.getBean("college", College.class);
```

What this means:

- "college" → **bean ID**
- `College.class` → expected type

Spring:

1. Looks for a bean named **college**
2. Confirms it is of type `College`
3. Returns the object

Usually defined as:

```
@Component
public class College {
    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

Bean name defaults to **class name with first letter lowercase**.

Calling business logic

```
college.principalName();
```

- Calls method on Spring-managed object
- Any dependencies inside `College` are:
 - Already injected
 - Already initialized

Execution flow

JVM starts

↓

main() method executes

↓

Spring container starts

↓

Configuration class loaded

↓

Beans created & wired

↓

getBean() fetches object

↓

Method executed

Spring Boot

Compare **your above Spring code** with **Spring Boot**, then we will cover **benefits** and **how to execute**.

Your CURRENT code (Plain Spring)

```
ApplicationContext context = new AnnotationConfigApplicationContext(CollegeConfig.class);
```

```
College college = context.getBean("college", College.class);
```

```
college.principalName();
```

What YOU do manually

- Create `ApplicationContext`
- Create `@Configuration` class
- Add `@ComponentScan`
- Manage dependencies
- Decide how the app starts

This is **pure Spring Framework**.

Above application using Spring Boot

Main Class

```
package com.testing.qe;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
```

```
@SpringBootApplication
public class TestApplication {

    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(TestApplication.class, args);
        College college = context.getBean(College.class);
        college.principalName();
    }
}
```

No `AnnotationConfigApplicationContext`

No `CollegeConfig` needed

College class (almost same)

```
package com.testing.qe;

import org.springframework.stereotype.Component;

@Component
public class College {
    public void principalName() {
        System.out.println("Principal is Dr. Sharma");
    }
}
```

What changed?

Plain Spring	Spring Boot
Manual config class	✗ Not needed
Manual component scan	✗ Auto
XML / Java config	✗ Optional
Embedded server	✗
Boilerplate code	✗

What `@SpringBootApplication` really doing?

`@SpringBootApplication`

This is **3 annotations combined**

`@Configuration`

`@EnableAutoConfiguration`

`@ComponentScan`

Meaning:

- `@Configuration` → Java config
- `@ComponentScan` → scans current package & subpackages
- `@EnableAutoConfiguration` → magic ✨ (auto setup)

BENEFITS of Spring Boot

✓ 1. No configuration headache

- No XML
- No manual bean wiring
- No server setup

✓ 2. Auto-configuration (HUGE)

Example:

- Add `spring-boot-starter-web`
- BOOM ☐
 - Tomcat configured
 - DispatcherServlet created
 - JSON mapper ready

You didn't write **anything**.

✓ 3. Embedded server

No need to install:

- Tomcat
- JBoss
- WebLogic

Spring Boot ships with **embedded Tomcat**.

✓ 4. Production-ready

Spring Boot gives:

- Health checks
- Metrics
- Logging
- Externalized config

How to EXECUTE Spring Boot app

Option 1: From IDE (IntelliJ / Eclipse)

- Right-click on `TestApplication`
- Click **Run**

✓ Most common

Option 2: Maven command

```
mvn spring-boot:run
```

Option 3: Run JAR (production way)

```
mvn clean package java -jar target/test-0.0.1-SNAPSHOT.jar
```

Project structure (Spring Boot way)

```
src/main/java
├── com.testing.qe
│   ├── TestApplication.java <-- main class
│   └── College.java
```

Main class must be at top package level

(or component scanning will fail)

Interview

Spring Boot = Spring + Auto Configuration + Embedded Server + Zero Boilerplate

When should YOU use Spring Boot?

- ✓ REST APIs
- ✓ Microservices
- ✓ Real-world projects
- ✓ Faster development

Environment in Spring

`org.springframework.core.env.Environment` is **one of the most important abstractions in Spring**, but it's often used without really understanding *why*.

What Environment actually is?

In **Spring Framework**, `Environment` is an **interface** that represents:

The current runtime environment of your application

It gives you **read-only access** to:

- Configuration **properties**
- Active **profiles**
- Default profiles

Why Spring needs `Environment`

Modern applications don't hardcode values like:

- Database URL
- Username/password
- Port numbers
- Feature flags

Instead, these values change by **environment**:

- dev
- test
- qa
- prod

`Environment` lets Spring **centralize and resolve** all of this configuration.

What can you do with `Environment`?

1. Read configuration properties

```
@Autowired
private Environment environment;

String url = environment.getProperty("spring.datasource.url");
```

This value could come from:

- `application.properties`
- `application.yml`
- OS environment variables
- JVM system properties
- Command-line arguments

You **don't** care where it came from — Spring resolves it.

2. Check which profile is active

```
if (environment.acceptsProfiles("prod")) {
    // production-only logic
}
```

Or:

```
String[] profiles = environment.getActiveProfiles();
```

This is extremely useful for:

- Logging
- Conditional behavior
- Feature toggles

3. Provide default values safely

```
String timeout =
    environment.getProperty("app.timeout", "30");
```

If `app.timeout` is missing → "30" is used.

No `NullPointerException`.

Why not just use `@Value`?

You *can* do this:

```
@Value("${server.port}")
private int port;
```

But `Environment` is better when:

Use case	Why <code>Environment</code> is better
Dynamic access	Can read properties anytime
Conditional logic	Profiles + properties together
Optional properties	Built-in defaults
Libraries/frameworks	Cannot rely on <code>@Value</code>

Typical real-world use cases

✓ Database switching

```
if (environment.acceptsProfiles("dev")) { // H2 database } else { // Oracle /
MySQL }
```

✓ Feature flags

```
if (environment.getProperty("feature.newUI", Boolean.class, false)) {
enableNewUI(); }
```

✓ Logging config

```
log.info("Active profile: {}", Arrays.toString(environment.getActiveProfiles()));
```

How Spring populates `Environment`

Spring collects properties from **PropertySources**, in this order:

1. Command-line args
2. JVM system properties
3. OS environment variables
4. `application.properties` / `yaml`
5. `@PropertySource`

You can inspect them:

```
ConfigurableEnvironment env = (ConfigurableEnvironment) environment;  
env.getPropertySources().forEach(ps -> System.out.println(ps.getName()));
```

Spring Boot vs Spring Framework

In **Spring Boot**:

- Environment is **auto-configured**
- `application.properties` is loaded automatically
- Profiles are first-class citizens

In plain Spring:

- You must define property sources manually

But the **Environment API is identical**.

When SHOULD you use `Environment`?

- ✓ Framework / infrastructure code
- ✓ Conditional configuration
- ✓ Reading optional properties
- ✓ Profile-based behaviour

When you should NOT

- ✗ Simple POJOs
- ✗ Business logic classes
- ✗ Where `@ConfigurationProperties` fits better

High level comparision

Aspect	Environment	@Value	@ConfigurationProperties 
Purpose	Programmatic access to config	Inject single value	Bind structured config
Style	Imperative (code-driven)	Declarative (annotation)	Declarative + type-safe
Type safety	✗ No	⚠ Partial	✓ Yes
Supports defaults	✓ Yes	⚠ Limited	✓ Yes
Best for	Framework / infra logic	Simple values	Application config
Testability	⚠ Medium	✗ Poor	✓ Excellent
Validation	✗ No	✗ No	✓ Yes (<code>@Validated</code>)

Hand on – Exercise to do

Create Project structure

```
demo-env/  
├─ build.gradle  
└─ src/  
    └─ main/  
        ├─ java/  
        │   └─ com/example/envdemo/  
        │       ├─ EnvDemoApplication.java  
        │       └─ EnvPrinter.java  
        └─ resources/  
            ├─ application.properties  
            ├─ application-dev.yml  
            └─ application-prod.yml
```

Gradle build file

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.2.1'  
    id 'io.spring.dependency-management' version '1.1.4'  
}  
  
group = 'com.example'  
version = '0.0.1-SNAPSHOT'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(17)  
    }  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter'  
}
```

Note: This uses **Spring Boot**, which auto-wires the `Environment`

application.properties (BASE configuration)

```
app.name=Env Demo App
app.timeout=30
app.source=application.properties
```

application-dev.yml (DEV profile)

```
app:
  timeout: 10
  source: application-dev.yml
```

Note: Overrides only when **dev** profile is active.

application-prod.yml (PROD profile)

```
app:
  timeout: 60
  source: application-prod.yml
```

Note: Overrides only when **prod** profile is active.

Main application class

```
package com.example.envdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EnvDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(EnvDemoApplication.class, args);
    }
}
```

Environment reader bean (MOST IMPORTANT PART)

```
package com.example.envdemo;

import jakarta.annotation.PostConstruct;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;

import java.util.Arrays;

@Component
public class EnvPrinter {

    private final Environment environment;

    public EnvPrinter(Environment environment) {
        this.environment = environment;
    }

    @PostConstruct
    public void print() {
        System.out.println("---- ENVIRONMENT DEMO ----");

        System.out.println("Active profiles: "
            + Arrays.toString(environment.getActiveProfiles()));

        System.out.println("app.name = "
            + environment.getProperty("app.name"));

        System.out.println("app.timeout = "
            + environment.getProperty("app.timeout"));

        System.out.println("app.source = "
            + environment.getProperty("app.source"));

        System.out.println("-----");
    }
}
```

Note: This bean runs **after the Environment is prepared**, but **before the app is fully ready**.

Run scenarios (THIS IS WHERE YOU LEARN)

Case 1: No profile (default)

bash

 Copy code

```
./gradlew bootRun
```

Output:

yaml

 Copy code

```
Active profiles: []
app.name = Env Demo App
app.timeout = 30
app.source = application.properties
```

✓ Only `application.properties` used

Case 2: DEV profile

bash

 Copy code

```
./gradlew bootRun --args='--spring.profiles.active=dev'
```

Output:

yaml

 Copy code

```
Active profiles: [dev]
app.name = Env Demo App
app.timeout = 10
app.source = application-dev.yml
```

✓ `application-dev.yml` overrides base

Case 3: PROD profile

bash

 Copy code

```
./gradlew bootRun --args='--spring.profiles.active=prod'
```

Output:

r

 Copy code

```
Active profiles: [prod]
app.name = Env Demo App
app.timeout = 60
app.source = application-prod.yml
```

Case 4: Command-line override (HIGHEST PRIORITY)

bash

 Copy code

```
./gradlew bootRun --args='--spring.profiles.active=dev --app.timeout=99'
```

Output:

yaml

 Copy code

```
Active profiles: [dev]
app.timeout = 99
```

✓ Command-line wins over everything

Property resolution order (MEMORIZE THIS)

From **highest** → **lowest** priority:

1. **Command-line arguments**
2. JVM system properties (`-D`)
3. OS environment variables
4. `application-{profile}.yml`
5. `application.properties`
6. Default values in code

That's why `--app.timeout=99` wins.

REST API - Application

Instead of calling:

```
college.principalName();
```

We will expose it as a **REST endpoint**:

```
GET http://localhost:8080/college/principal
```

and get a response like:

```
Principal is Dr. Sharma
```

Create a Spring Boot project

Use: IntelliJ / Eclipse **OR**

Required dependency

Add only **Spring Web**

(Maven dependency)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This is powered by **Spring Boot**, built on **Spring Framework**.

Project structure (IMPORTANT)

```
src/main/java
├── com.testing.qe
│   ├── TestApplication.java ← main class
│   ├── College.java         ← business logic
│   └── CollegeController.java ← REST API
```

Main class must be at **top package level**.

Main Application class

```
package com.testing.qe;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class TestApplication {

    public static void main(String[] args) {
        SpringApplication.run(TestApplication.class, args);
    }
}
```

✓ No `ApplicationContext`

✓ No config class

✓ No manual wiring

Business logic (same as before)

```
College.java
```

```
package com.testing.qe;
```

```
import org.springframework.stereotype.Component;
```

```
@Component
public class College {

    public String principalName() {
        return "Principal is Dr. Sharma";
    }
}
```

✓ Spring manages this bean

✓ No `new College()` anywhere

REST Controller (NEW)

CollegeController.java

```
package com.testing.qe;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class CollegeController {

    @Autowired
    private College college;

    @GetMapping("/college/principal")
    public String getPrincipalName() {
        return college.principalName();
    }
}
```

What annotations mean (VERY IMPORTANT)

Annotation	Purpose
@RestController	Marks class as REST API
@GetMapping	Maps HTTP GET request
@Autowired	Injects <code>College</code> bean
@Component	Makes class a Spring bean
@SpringBootApplication	Auto config + scan

How execution works (FLOW)

Browser / Postman

|
| GET /college/principal
v

CollegeController

|
v

College (business logic)

|
v

Response returned

How to RUN the application

✓ Option 1: From IDE

- Right-click `TestApplication`
- Click **Run**

✓ Option 2: Maven

```
mvn spring-boot:run
```

✓ Option 3: JAR (production style)

```
mvn clean package java -jar target/*.jar
```

How to TEST the REST API

In browser

<http://localhost:8080/college/principal>

In curl

```
curl http://localhost:8080/college/principal
```

Why REST API is better than `main()` call

Old way

Console output

Local JVM

Not reusable

Hard to scale

REST way

HTTP response

Network accessible

Used by UI / Mobile

Cloud-ready

Interview one-liner

Spring Boot REST API exposes business logic using HTTP endpoints, making applications scalable, reusable, and platform-independent.

What next

- Convert response to **JSON**
- Add **POST API**
- Add **DTO**
- Connect to **Oracle DB**
- Explain **@RestController** vs **@Controller**

DB Access

Spring Boot manages the connection pool for your project

You should **never manually open/close connections everywhere**.

Layers in accessing a Database

Controller → Service → Repository (DAO) → Database

- **Controller** → handles HTTP requests (optional if not REST)
- **Service** → business logic
- **Repository (DAO)** → all SQL (SELECT / UPDATE / DELETE)
- **DataSource** → manages JDBC connections (Spring handles this)

Handle DB connection

Use **DataSource** + **JdbcTemplate**

Spring Boot auto-configures this from Properties file

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521/orclpdb
spring.datasource.username=app_user
spring.datasource.password=secret
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
```

Package Structure

This follows **Java + Spring standards**.

```
com.example.college
├── controller
│   └── StudentController.java
├── service
│   └── StudentService.java
├── repository
│   └── StudentRepository.java
├── model
│   └── Student.java
└── CollegeApplication.java
```

CollegeApplication

```

package com.example.college;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CollegeApplication {

    public static void main(String[] args) {
        SpringApplication.run(CollegeApplication.class, args);
    }
}

```

Controller

optional, if REST API

```

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping
    public List<Student> getAll() {
        return service.getStudents();
    }
}

```

Model Class

- ✓ No DB code
- ✓ Just data

```

package com.example.college.model;

public class Student {

    private Long id;
    private String name;
    private Integer age;

    // getters and setters
}

```

Repository layer (ALL SQL goes here)

- One class = **one table**
- All SQL in **one place**
- Uses **prepared statements automatically**
- No connection handling code

Service layer (business logic)

- ✓ No SQL
- ✓ Easy to test
- ✓ Business rules go here

```

package com.example.college.repository;
import com.example.college.model.Student;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public class StudentRepository {
    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    // SELECT
    public List<Student> findAll() {
        String sql = "SELECT id, name, age FROM student";
        return jdbcTemplate.query(
            sql,
            (rs, rowNum) -> {
                Student s = new Student();
                s.setId(rs.getLong("id"));
                s.setName(rs.getString("name"));
                s.setAge(rs.getInt("age"));
                return s;
            }
        );
    }

    // INSERT
    public int save(Student student) {
        String sql = "INSERT INTO student (id, name, age) VALUES (?, ?, ?)";
        return jdbcTemplate.update(sql,
            student.getId(),
            student.getName(),
            student.getAge());
    }

    // UPDATE
    public int updateName(Long id, String name) {
        String sql = "UPDATE student SET name = ? WHERE id = ?";
        return jdbcTemplate.update(sql, name, id);
    }

    // DELETE
    public int deleteById(Long id) {
        String sql = "DELETE FROM student WHERE id = ?";
        return jdbcTemplate.update(sql, id);
    }
}

```

```

package com.example.college.service;
import com.example.college.model.Student;
import com.example.college.repository.StudentRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class StudentService {
    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<Student> getStudents() {
        return repository.findAll();
    }

    public void createStudent(Student student) {
        repository.save(student);
    }

    public void updateStudentName(Long id, String name) {
        repository.updateName(id, name);
    }

    public void deleteStudent(Long id) {
        repository.deleteById(id);
    }
}

```